

Design V5

Architectural Framework and Shared Pipeline Design

Equivariant 3D Fracture Reassembly

M.Sc. Thesis — Design Document

Revision 5, superseding Design V4

This revision replaces Design V4. Every figure quoted here has been measured on the implemented system or on the Breaking Bad dataset itself; where a quantity is still an assumption, it is labelled as one. Sections that depart from V4 carry an explicit note saying what changed and why, so that a reader of V4 can find the deltas without re-reading the whole document.

Contents

1	Architectural Framework and Shared Pipeline Design	3
1.1	Introduction	3
1.1.1	Scope of this revision	3
1.2	The dataset, measured	3
1.3	Geometric feature representation and graph construction	4
1.3.1	Requirements	4
1.3.2	Fragment centralisation and scene standardisation	5
1.3.3	Node features	5
1.3.4	Edge features	5
1.3.5	Fragment scale as an invariant gate	5
1.3.6	Connectivity	6
1.4	Equivariance: the constraint that shapes the design	6
1.4.1	Definition	6
1.4.2	Equivariant to its own pose, invariant to every other fragment’s	6
1.4.3	What may cross the gap	6
1.4.4	A consequence: the perturbation cancels from the error	7
1.4.5	Numerical verification	7
1.5	Fracture surfaces and inter-fragment tokens	8
1.5.1	Why the virtual-node design was withdrawn	8
1.5.2	Two labellings, and the inference gap	8
1.5.3	Token selection: geodesic-disk sampling under a per-scene budget	9
1.5.4	The cost of the resulting graph	9
1.6	Network architecture	9
1.6.1	Vector Neuron primitives	9
1.6.2	Intra-fragment attention	10
1.6.3	Cross-fragment attention	10
1.6.4	Layer schedule and propagation reach	10
1.6.5	Rotation head	11
1.7	Training objective	11
1.7.1	Rotation loss	11
1.7.2	Geometric auxiliary losses	12
1.7.3	Interface correspondence: two corrections	12
1.7.4	Composite objective	13
1.8	Reference values	13
1.9	Correspondence discovery	14
1.10	Geometric optimisation-based translation recovery	14
1.11	Workflow: training, validation, and inference	14
1.11.1	The shared path	14
1.11.2	Validation is not label-free	15
1.11.3	A real broken object	15

1.12	Implementation within a commodity hardware budget	15
1.13	Status, and what has not been established	16
1.13.1	Established	16
1.13.2	Not established	16
1.13.3	Open questions	16

Chapter 1

Architectural Framework and Shared Pipeline Design

1.1 Introduction

The objective of this research is to reconstruct fractured three-dimensional objects from a collection of disconnected fragments. Unlike conventional shape reconstruction, fragment reassembly requires reasoning simultaneously about local geometry, global object structure, and relationships between fragments that no longer share explicit graph connections.

The framework treats every fragment as an independent graph and employs an equivariant graph neural network to learn geometric representations that remain consistent under rotation. The central hypothesis is unchanged from V4: reassembly should be decomposed into two problems. First, the network learns how fragments are oriented. Second, once orientations are recovered, translations are solved by geometric optimisation rather than direct regression. This lets the network concentrate on geometric relationships and delegates final placement to a mathematically grounded procedure.

The research question that motivates the whole design is one of *economy*. The current state of the art, GARF, reaches 6.1° rotation RMSE using four H100 accelerators for 72 hours. This work asks whether equivariance can substitute for scale: whether a network that is $SO(3)$ -equivariant *by construction* needs to see far fewer examples than one that must learn rotational invariance from data, and can therefore be trained within a two-GPU, twelve-hour-session budget on Tesla T4 hardware.

1.1.1 Scope of this revision

V4 proposed two candidate backbones, Vector Neuron Graph Attention (VN-GAT) and Tensor Field Graph Attention (TF-GAT). Only VN-GAT has been implemented and is described here. TF-GAT remains a documented alternative that was not pursued; nothing in the surrounding pipeline would prevent substituting it later.

Table 1.1 summarises what has changed. Each row is expanded in the section named.

1.2 The dataset, measured

Several V4 decisions were made against assumed properties of the data. An exhaustive pass over the corpus was therefore run before finalising this revision: 1,442 objects and **1,096,825 fragments**, processed in approximately one hour on four workers.

Component	Design V4	Design V5
Inter-fragment communication	8 virtual nodes per fragment, pooled and broadcast	Direct attention between sampled fracture-surface vertices (§1.5, §1.6.3)
Edge features	length, midpoint, two face normals	two face normals, relative position (§1.3.4)
Fragment scale	implicit	explicit invariant gate (§1.3.5)
Connectivity	adjacency and incidence	adjacency only (§1.3.6)
Equivariance condition	$L(R \cdot f) = D(R) L(f)$	additionally invariant to <i>other</i> fragments’ poses (§1.4)
Rotation representation	normalised quaternion	6D vector pair, Gram–Schmidt (§1.6.5)
Rotation loss	arccos of the trace	atan2 formulation (§1.7.1)
Interface loss	$\ \sum_i z_i\ ^2$	InfoNCE over coincidence clusters (§1.7.3)
Midpoint loss	present	removed with the midpoint feature (§1.7.2)
Reference values	not specified	every term has a measured chance value (§1.8)

Table 1.1: Revisions from Design V4. Three of these — the interface loss, the rotation loss, and the equivariance condition — are corrections of defects rather than refinements.

Quantity	Median	Mean	Maximum
Vertices per fragment	296	1,550	83,039
Fracture vertices per fragment	125	164	3,587
Fragments per scene	3	8	99

Table 1.2: Breaking Bad, measured over the full corpus using coincidence ground truth.

Dataset-wide, the fracture surface is **10.6% of vertices, 6.2% of faces and 7.6% of edges**. Two reduction figures are reported throughout this document because they answer different questions and diverge sharply: the *per-fragment mean* removes 49.9% of vertices, weighting every fragment equally, while the *dataset aggregate* removes 89.4%, weighting by size. Most fragments are small, and small fragments are mostly fracture surface, so a typical fragment loses half its vertices while the dataset loses nine tenths.

Two consequences shape the design. The heavy tail — a maximum of 83,039 vertices against a median of 296 — means any per-fragment memory budget must be set by the tail, not the median. And because the true fracture surface is small, the inter-fragment graph is far cheaper than V4 assumed (§1.5.4).

1.3 Geometric feature representation and graph construction

1.3.1 Requirements

Meshes define graph structures naturally: vertices are nodes, mesh edges are graph edges. The representation must preserve geometric information, remain compatible with equivariant architectures, and carry information useful for fracture matching.

The compatibility requirement is more restrictive than it appears. Vector Neuron features are shaped $(C, 3)$, and a rotation acts on the last axis as $x \mapsto xR^\top$. Anything without a direction — a length, a scalar scale, a count — has no legal slot in that tensor, and placing one there silently breaks equivariance. This single constraint drives most of the changes in this section.

1.3.2 Fragment centralisation and scene standardisation

Fragments occupy arbitrary locations. Using absolute coordinates would let the network learn dataset placement biases rather than intrinsic geometry. For each fragment f_k with vertex set V_k , let

$$c_k = \frac{1}{|V_k|} \sum_{v \in V_k} x_v, \quad \tilde{x}_v = x_v - c_k.$$

Changed since V4. Centralisation is now followed by a per-scene standardisation. All fragments of a scene are divided by a single scene radius, so the network sees shapes at a consistent scale. One divisor is used for both the ground-truth and the perturbed copy, because a rotation does not change a radius — which is what allows the supervision relation of §1.7.1 to survive normalisation exactly.

Centralisation must remain *reversible*: stage two recovers per-fragment translation, and the original centroid vectors have to be retrievable.

1.3.3 Node features

Each node corresponds to a mesh vertex. The node representation is a vector-valued pair

$$V_v = [\tilde{x}_v, n_v] \in \mathbb{R}^{2 \times 3},$$

the centralised coordinate and the vertex normal. The coordinate describes local shape within the fragment; the normal describes local surface orientation. This is unchanged from V4.

1.3.4 Edge features

For an edge joining u and v , V4 used $e_{uv} = [l_{uv}, m_{uv}, n_{uv}^{(1)}, n_{uv}^{(2)}]$, combining a scalar length, a midpoint, and two face normals.

Changed since V4. The length and the midpoint are removed; a relative position is added. The edge representation is now three vector channels,

$$e_{uv} = [n_{uv}^{(1)}, n_{uv}^{(2)}, x_u - x_v] \in \mathbb{R}^{3 \times 3}.$$

The length was a scalar sharing a tensor with directions, which is exactly the violation described in §1.3.1. Its content — local scale — is recovered by the relative position $x_u - x_v$, whose norm *is* the length and which additionally carries direction. The midpoint duplicated information already present in the two endpoint coordinates.

The two face normals are stored in a canonical order derived from the mesh orientation, so that the pair is a well-defined function of the edge rather than of the order in which faces happened to be enumerated. Only these two channels participate in the face-normal loss; the relative position must not, because a cosine loss would normalise away the very length that is its content.

1.3.5 Fragment scale as an invariant gate

Fragments span four to 83,039 vertices, so raw scale is heavy-tailed and would dominate anything it were concatenated with. Scale has no direction, so it cannot join the vector stream. It conditions that stream multiplicatively instead:

$$g = \text{softplus}(\text{MLP}([\text{Gram}(x), \log s])), \quad x \leftarrow g \odot x,$$

where $\text{Gram}(x)$ denotes the matrix of pairwise inner products of the vector channels, which is invariant, and s is the fragment scale. The gain g is invariant, so the product remains equivariant. The logarithm is used because of the heavy tail; GARF applies a positional encoding for the same reason.

1.3.6 Connectivity

Changed since V4. V4 retained both an adjacency (COO) structure and an incidence structure pairing nodes with globally shifted edge identifiers. Only the adjacency structure is retained.

Edges are stored bidirectionally, so structural symmetry is preserved. The incidence structure existed to let messages pass between nodes and edges; in the implemented design, edge features are consumed directly by the attention mechanism at each edge, which serves the same purpose without a second index tensor to keep synchronised.

A correctness requirement that is easy to miss: the ground-truth and perturbed copies of a scene must share *one* topology. The losses match vertices, edges and normals by index, so two independently constructed copies would compare unrelated rows without raising an error. Face de-duplication reorders faces lexicographically and can produce two different vertex orderings, so the two copies are built once and split, and their edge indices are asserted equal.

1.4 Equivariance: the constraint that shapes the design

1.4.1 Definition

Let $R \in \text{SO}(3)$. A mapping L is equivariant when

$$L(R \cdot f_k) = D(R) L(f_k),$$

where $D(R)$ acts on the output space. A conventional network treats two rotated copies of one fragment as unrelated inputs and must learn the same geometry many times over; an equivariant layer obtains the rotation for free, as an algebraic property rather than as something acquired from augmented data.

1.4.2 Equivariant to its own pose, invariant to every other fragment's

Changed since V4. V4 stated only the equation above. It is necessary but not sufficient, and the missing half determines the entire inter-fragment design.

Each fragment is perturbed by its *own* rotation. The label for fragment i therefore does not change when fragment j lands at a different angle — and the prediction must not either. Writing A for a rotation applied to a single fragment's input, fragment i 's representation H_i must satisfy both lines:

$$\begin{aligned} H_i(P_1, \dots, P_i A, \dots, P_N) &= H_i(\cdot) A && \text{equivariant to its own pose,} \\ H_i(P_1, \dots, P_j A, \dots, P_N) &= H_i(\cdot) && \text{invariant to every other pose } (j \neq i). \end{aligned}$$

This is equation 7 of Wu et al., *Leveraging SE(3) Equivariance for Learning 3D Geometric Shape Assembly*. The second line is forced by the task, before any architecture is chosen. An architecture that can see fragment j 's orientation must *learn* to ignore it — from data, by augmentation — which is precisely what equivariance is supposed to make unnecessary. Worse, j 's orientation is not weak signal but pure noise injected by the perturbation, feeding directly into i 's prediction.

1.4.3 What may cross the gap

The constraint is not “no vectors between fragments”. The cross-fragment layer takes $(T, C, 3)$ vector features and returns $(T, C, 3)$ vector features, and every operation on the spatial axis is equivariant. What is constrained is what a message may *depend on*.

Wu et al.’s correlation module is the construction that satisfies both lines: $C_{ij} = G_j \cdot F_i$, where G_j is an *invariant matrix* describing fragment j and F_i is fragment i ’s *own equivariant* features. A matrix, not a scalar — the sender may re-mix the receiver’s channels arbitrarily. What it cannot do is contribute a direction of its own, because a direction carries an orientation.

What crosses is *shape*. What does not cross is *pose*. Nothing is lost by that: the relative pose of two arbitrarily tumbled fragments is the perturbation, not the object.

A corollary constrains the attention scores. The natural score $\langle W_q x_i, W_k x_j \rangle$ is unusable, because under independent perturbations it becomes $\langle A_i a, A_j b \rangle$, which depends on the relative rotation $A_i^\top A_j$ — exactly the quantity that is noise. Scores are therefore inner products of each token’s invariant description.

1.4.4 A consequence: the perturbation cancels from the error

This property has a consequence for reading training curves that is worth stating explicitly, because the obvious interpretation of an initial loss is wrong.

The head returns $R_{\text{pred}} = M^\top$, where M is the equivariant frame, and the frame co-rotates with its input. On a fragment perturbed by Q the prediction is $M_{\text{asm}}^\top Q^\top$ while the label is $Q^\top$, so the error rotation is

$$R_{\text{pred}} R_{\text{label}}^\top = M_{\text{asm}}^\top Q^\top Q = M_{\text{asm}}^\top.$$

The perturbation cancels exactly. An untrained model’s rotation error is the geodesic angle of its own frame on the *assembled* fragment, and has nothing to do with how that fragment was tumbled. Verified numerically: across three independent perturbations the per-fragment errors agreed to 3×10^{-13} **radians**.

Two things follow. First, 126.5° is not a distribution an untrained equivariant model draws from; it is the mean angle between two *independent* uniform rotations. Here the error is a deterministic function of the untrained network, so it equals 126.5° only if those frames happen to be uniformly spread. Measured over twelve seeds on a 24-fragment batch, the initial error was $132.3^\circ \pm 11.8$ for one architecture and $121.9^\circ \pm 15.0$ for another — *architecture-dependent*, not a defect.

Second, it restates the objective usefully. Since the initial error *is* $\angle M_{\text{asm}}$, training is literally driving the frame on assembled fragments to the identity. That is a fixed, learnable target, because the assembled pose is a convention shared by every scene in the dataset.

1.4.5 Numerical verification

Equivariance is an algebraic identity, so it should hold to machine precision rather than approximately. All checks below were run in double precision.

Two defects were caught by these checks, both of the kind that trains without complaining. A near-identity initialisation implemented by zeroing a gate’s final weight makes the layer’s output constant, which zeroes the gradient into everything upstream — 31 of 77 parameters dead on the first steps, waking only as a bias drifted. And $\sqrt{s + \epsilon}$ perturbs *every* norm rather than only the near-zero ones: the head came out orthonormal to only 10^{-8} with $\det = 0.99999995$. Clamping the squared sum instead is exact away from zero.

Property	Residual
Every Vector Neuron primitive, $L(xR^\top) = L(x)R^\top$	$\leq 10^{-15}$
Gram–Schmidt head orthonormal, $\det = +1$	exact to 10^{-16}
Intra-fragment attention, equivariance	1.3×10^{-15}
Cross-fragment, own-pose equivariance	4.4×10^{-16}
Cross-fragment, other-pose <i>invariance</i>	3.3×10^{-16}
Cross-scene leakage within a batch	exactly 0
Whole model, per-fragment equivariance	$\leq 10^{-9}$
Perturbation cancellation (§1.4.4)	3×10^{-13}

Table 1.3: Measured equivariance residuals. A failure at 10^{-6} would indicate a real break rather than accumulated round-off.

1.5 Fracture surfaces and inter-fragment tokens

1.5.1 Why the virtual-node design was withdrawn

V4 introduced eight coordinate-aware virtual nodes per fragment, pooled upward from vertices, attended across fragments, and broadcast back down. That design is withdrawn for two reasons.

The first is the constraint of §1.4.2. Virtual nodes with fixed spatial coordinates are *positions*, and pooling vertex features onto them and attending between them lets a fragment’s orientation influence its neighbours’ representations. The mechanism cannot satisfy the invariance line without additional machinery.

The second is empirical. The earlier VN-GAT design reached approximately 43° on training data but stalled near 94° on validation — close to the *axis-only floor* of 89.9° , which is the score of a model that recovers a fragment’s axis but not its rotation about that axis. That is the signature of a per-fragment canonicaliser with no usable access to inter-fragment relationships.

Changed since V4. Virtual nodes are replaced by direct attention between vertices lying on fracture surfaces. All vertices are retained as graph nodes; the fracture mask decides only which vertices are *eligible* to become cross-fragment tokens.

1.5.2 Two labellings, and the inference gap

A fragment’s surface is two things glued together: the object’s original exterior, which is smooth and shared with no one, and the fracture surface, where it tore away and which matches exactly one other fragment. Two independent labellings are available.

Method	What it uses	Available at inference?
Dihedral	sharp dihedral variation within one fragment	yes, one fragment alone
Coincidence	vertices another fragment also occupies	no, needs the assembled scene

Coincidence is ground truth and is how supervision is derived. Dihedral is the estimate a model can compute at test time, and it is **much weaker than V4 assumed**: at its tuned threshold it over-labels by approximately $3.9\times$, with precision 0.24 against recall 0.93. It also lights up handles, rims and feet — sharp because the object was *designed* that way, not because it broke there.

This is the single largest gap between the reported results and a real deployment, and §1.13 states it as such. The threshold is chosen by measurement rather than by eye, scoring dihedral against coincidence across a sweep and reporting recall-weighted F_β rather than F_1 : a missed

fracture face is an edge the model never sees, while a spurious one is only noise, so the two error types are not symmetric.

1.5.3 Token selection: geodesic-disk sampling under a per-scene budget

Cross-fragment tokens are selected from the masked vertices by farthest-point sampling under a *geodesic* metric — shortest path along fracture-surface mesh edges — rather than a Euclidean one. The distinction matters on folded or re-entrant surfaces, where two points may be close in space while lying far apart on the surface; Euclidean sampling treats them as redundant and leaves part of the interface unsampled.

The budget is *per scene*, not per fragment. A single total T is divided among the fragments in proportion to their fracture-surface area, and sampling then proceeds within each fragment’s own eligible set. Two properties follow.

A per-fragment cap is deliberately *not* imposed. If a scene contains one large fragment and several small ones — a head with the ears, nose and part of the hair broken off — proportional allocation gives the large piece more tokens, which is the intended behaviour. Capping would destroy that proportionality for no benefit, because the total alone already bounds the cost: for any split $\{t_i\}$ summing to T , the number of undirected cross-fragment pairs is

$$\frac{T^2 - \sum_i t_i^2}{2} < \frac{T^2}{2}.$$

Sampling is per fragment and never over the pooled scene. At input time the fragments sit in arbitrary perturbed poses, so a single global farthest-point pass would select points according to where the pieces happened to land — a different sample for every perturbation of the same scene, and no equivariance left.

1.5.4 The cost of the resulting graph

V4’s virtual-node compression was motivated by an assumed cost. With the true fracture surface measured at 10.6% of vertices, connecting every fracture vertex to every fracture vertex of every *other* fragment turns out to be affordable with no reduction at all.

Fragments in scene	Cross-fragment pairs	vs GARF’s 6-layer stack
3 (median scene)	46,875	0.001×
8 (mean scene)	753,088	0.01×
20	2,968,750	0.04×
99 (largest in corpus)	75,796,875	1.01×

Table 1.4: Unreduced inter-fragment graph cost. It crosses GARF’s whole stack only at 99 fragments, which is exactly the largest scene in the dataset.

Reduction is therefore not needed for cost. Sampling is retained for a different reason: to make the inter-fragment connection count *fixed and predictable* rather than a function of whatever mesh happens to arrive. A retraction is owed here, and is recorded in §1.13.

1.6 Network architecture

1.6.1 Vector Neuron primitives

Features are $(C, 3)$ tensors. The primitives are bias-free linear channel mixing, a learned-hyperplane leaky rectifier, a layer normalisation over invariant magnitudes, and an invariant

readout formed from inner products with learned directions. A linear map with a bias would break equivariance, because a constant offset does not rotate; the absence of bias is structural, not stylistic.

1.6.2 Intra-fragment attention

Attention along real mesh edges. Scores are invariant inner products of projected query and key features, so weighting equivariant messages by them leaves the result equivariant. Each node also carries a self-transform outside the softmax, so a node with no incident edges still has an output, and so the layer can learn to ignore its neighbourhood entirely.

Two implementation properties are worth recording because they are not cosmetic. Because the linear maps have no bias, $W[a; b; c] = W_a a + W_b b + W_c c$ exactly, so concatenation can be replaced by separate projections that are summed. This removes the concatenation temporaries and, more importantly, allows each projection to run on the N nodes before being gathered to the E edges rather than after — roughly one sixth of the work on a triangle mesh.

1.6.3 Cross-fragment attention

The layer that replaces V4’s virtual nodes. It is a Vector Neuron layer throughout, and it implements the correlation construction of §1.4.3:

$$\begin{aligned} \text{logits}_{ij} &= \langle q(\text{inv}_i), k(\text{inv}_j) \rangle / \sqrt{d} && \text{invariant to both poses,} \\ \alpha &= \text{softmax}_j \text{ within the scene, excluding } j\text{'s own fragment,} \\ \text{out}_i &= x_i + \left(\sum_j \alpha_{ij} G_j \right) x_i. \end{aligned}$$

G_j is a per-head channel-mixing matrix read from fragment j ’s invariants. Because G is a *linear* function of those invariants, $\sum_j \alpha_{ij} G(v_j) = G(\sum_j \alpha_{ij} v_j)$, so the aggregation happens on the small invariant vectors and the matrix is built once per query rather than once per pair. This is not a cosmetic rearrangement: a per-pair matrix would be gigabytes at the pair counts a large scene reaches.

Tokens attend across fragments *within one scene*, never inside their own fragment — the mesh graph already covers that — and never across scenes. The cross-scene case is not hypothetical: a batch concatenates unrelated objects and fragment identifiers are globally unique, so an unmasked layer silently lets one object inform another and makes the prediction depend on batch composition. That defect occurred during development and is now excluded by construction and asserted in the test suite.

1.6.4 Layer schedule and propagation reach

The backbone is five intra-fragment layers, three cross-fragment layers, then one further intra-fragment layer: describe each fragment, let the fragments talk, then spread what they heard. The final clause is not decoration.

A cross layer writes only to *token* vertices, and the rotation head pools a mean over *every* vertex — so a vertex the tokens never reach contributes to the prediction without having heard from another fragment. Each intra layer after the last cross layer buys exactly one hop along mesh edges. Measured on real scenes:

The reach is measured on the actual meshes at start-up rather than assumed, because it depends on how the fracture surface is shaped and cannot be derived from the token count alone.

After the last cross layer	Vertices informed
nothing	17%
one intra layer	49%
two intra layers	64%

Table 1.5: Fraction of vertices within reach of a cross-fragment token. The first trailing layer buys 32 points, the second only 15 — tokens cluster on fracture surfaces, so their neighbourhoods overlap.

1.6.5 Rotation head

Changed since V4. V4 predicted a normalised quaternion. V5 predicts two equivariant 3-vectors and orthonormalises them by Gram–Schmidt.

The quaternion was chosen in V4 to avoid gimbal lock. It does, but it shares with every three- and four-parameter parameterisation a subtler problem: the map from $\text{SO}(3)$ to the representation is discontinuous, so a network must learn a discontinuous function, and the antipodal identification $q \sim -q$ leaves the target ambiguous unless handled explicitly. The 6D representation is continuous, which is the property that matters for regression.

More decisively, it fits the equivariant setting exactly: the two vectors are ordinary vector features, so they rotate with the input for free, and Gram–Schmidt is equivariant. The head therefore produces an equivariant frame M with no additional constraint, and returns $R_{\text{pred}} = M^\top$ (the *rows*, not the columns). The derivation, and the reason this transpose is the single easiest thing in the system to get backwards, are given in §1.7.1.

1.7 Training objective

1.7.1 Rotation loss

Centred, the perturbation satisfies $v_{\text{pert}} = v_{\text{gt}} Q^\top$. Vector features inherit that rotation, so the frame satisfies $M_{\text{pert}} = Q M_{\text{gt}}$. The label is the rotation that undoes the perturbation, $R_{\text{gt}} = Q^\top$, so the head returns $R_{\text{pred}} = M^\top$, giving $R_{\text{pred}} = M_{\text{gt}}^\top Q^\top$ — correct exactly when the network maps an assembled fragment to the identity frame.

Getting this transpose backwards is invisible at initialisation, because chance is chance in either direction, and would surface only as a model that never converges. It is therefore asserted directly against a known perturbation rather than inferred from a loss curve.

Changed since V4. V4 defined the angular error as $\theta = \arccos((\text{tr}(R_k^\top R_{\text{gt}}) - 1)/2)$. V5 uses an atan2 formulation of the same angle.

The two agree in value and differ in gradient. The derivative of $\arccos$ is unbounded at its endpoints, so the gradient of V4’s loss *diverges exactly as the model converges*, swamping every other term at precisely the point where fine alignment matters. The atan2 form, taking the angle from the norm of the axis component against the trace, is finite throughout. Its guarded norm also takes a far smaller floor, because atan2 ’s derivative cancels the norm’s — the loss is exactly zero at a perfect fit rather than reaching a 5×10^{-9} radian floor.

1.7.2 Geometric auxiliary losses

The predicted rotation is applied to the centred, non-standardised fragment and compared against the ground-truth copy, index by index:

$$L_{\text{pos}} = \frac{1}{|V|} \sum_v \|x_v - \hat{x}_v\|_2^2, \quad L_{\text{normal}} = \frac{1}{|V|} \sum_v (1 - n_v \cdot \hat{n}_v),$$

$$L_{\text{face}} = \frac{1}{|E|} \sum_e \left[(1 - n_e^{(1)} \cdot \hat{n}_e^{(1)}) + (1 - n_e^{(2)} \cdot \hat{n}_e^{(2)}) \right].$$

Changed since V4. V4’s edge-midpoint loss L_{mid} is removed together with the midpoint feature.

The face term is restricted to the two normal channels inside the loss rather than by the caller. Edges carry three channels, and passing all three is the obvious mistake: a cosine loss would normalise the relative position, whose *length* is its content, and would silently move the term’s chance value from 2.0 to 3.0.

1.7.3 Interface correspondence: two corrections

This is the term that has changed most, and both changes are corrections.

V4’s formulation is minimised by cancellation. V4 defined

$$L_{\text{emb-v}} = \sum_{x \in X_{\text{shared}}} \left\| \sum_{(f,v) \in C_V(x)} z_v^{(f)} \right\|_2^2.$$

Two *opposite* embeddings score zero; two *identical* ones score $4\|z\|^2$. The objective rewards exactly the wrong configuration while looking like a reasonable consistency penalty.

The obvious repair is minimised by collapse. Replacing the norm of the sum with the scatter about each cluster centroid,

$$L = \text{mean}_c \text{mean}_{i \in c} \|z_i - \bar{z}_c\|^2,$$

fixes the cancellation and leaves a second trivial minimum: a *constant* embedding scores exactly zero. This is not hypothetical — it is what the first real training run did. Over 180 steps the loss fell from 0.0109 to 0.0010 while the spread of the embeddings fell from 0.166 to 0.021 and their norm held at 2.0. The head was converging on a single vector and reporting success the whole way down. A constant embedding is worse than an untrained one for what this head exists to do, since stage two matches by mutual nearest neighbours and a constant embedding ties every distance.

Changed since V4. The term is now InfoNCE over the coincidence clusters:

$$L_{\text{corr}} = -\text{mean}_i \log \frac{\sum_{p \in \text{pos}(i)} \exp(s_{ip}/\tau)}{\sum_{k \neq i} \exp(s_{ik}/\tau)},$$

with s the cosine similarity between ℓ_2 -normalised embeddings.

It is written contrastively rather than as a variance floor because the negatives are exactly the confusion set at matching time — the other fracture vertices of the same scene — so the training objective and the downstream use become the same question. Collapse now scores $\log(A-1) - \log|\text{pos}|$, verified against the closed form: **4.844** for 128 anchors in pairs, against **0.024** for a perfect embedding. The point is not that collapse became impossible, but that it became *visible*.

Anchors are capped per batch, because the similarity matrix is quadratic in them and a scene can label thousands of vertices.

A reporting consequence: the composite total no longer bottoms out at zero, since InfoNCE retains a small temperature-dependent floor even for a perfect embedding. The number to read instead is **match@1**, the fraction of coincident vertices whose true partner is their own nearest neighbour. It runs from near zero at chance to exactly 1, needs no per-batch reference, and a collapsed embedding cannot fake it.

1.7.4 Composite objective

$$L_{\text{total}} = \lambda_1 L_{\text{rot}} + \lambda_2 L_{\text{pos}} + \lambda_3 L_{\text{normal}} + \lambda_4 L_{\text{face}} + \lambda_5 L_{\text{corr}}.$$

All weights are currently 1.0 and *deliberately untuned*. The terms have very different natural scales — radians, a cosine in $[0, 2]$, a distance in normalised units, a log-ratio — and choosing weights before observing how each one moves is guesswork presented as a decision. Every term is reported separately so that the first training runs *measure* the relative scales rather than assume them.

1.8 Reference values

Changed since V4. V4 specified no reference values. Every number in this system is now read against what it must be when the model has learned nothing.

A geometric loss that is silently wrong still descends, so “the loss went down” proves nothing. All values below were established by Monte Carlo with fixed seeds, not asserted.

Quantity	At chance	Meaning
Geodesic angle	126.48°	$\pi/2 + 2/\pi$; a model here has learned nothing
Euler RMSE, random	86.29°	an independent uniform prediction
Euler RMSE, identity	83.14°	always predicting no rotation
Axis-only floor	89.9°	axis recovered, rotation about it not
Node normal loss	1.0	mean of $1 - \cos$ over random directions
Face normal loss	2.0	two normals per edge, hence twice
Node position loss	4/3	on a unit sphere

Table 1.6: Reference values. Every term of the objective has one.

The two Euler rows are the important pair, and the ambiguity in the word “chance” is not harmless. A model that collapses to predicting the identity — the cheapest way to reduce a rotation loss early in training — scores 83.14°, which is *three degrees better than guessing*, on the metric GARF reports as its headline. The geodesic angle is identical for both, 126.48°, because a uniform rotation’s angle is distributed the same way whether measured against the identity or against another uniform rotation. The geodesic angle therefore does not pay for the collapse, which is the argument for reporting it as the primary number and Euler RMSE only for comparability.

The initial rotation reading is checked against a deliberately wide band, for the reason established in §1.4.4: it is the untrained frame’s own angle and moves with the architecture. A band tight around 126.5° would flag every architecture change as a defect. What the check still catches is gross breakage — a model beginning near 0° or 180° cannot be measuring the angle it claims to.

1.9 Correspondence discovery

After training, interface embeddings identify candidate matching points between fragments. For every interface point a nearest-neighbour search is performed in embedding space, and *mutual* nearest-neighbour matching is required: two points are a correspondence only if each selects the other. This reduces false matches and provides reliable geometric constraints for the translation solver. This section is unchanged from V4 in principle; what has changed is that the embedding it operates on is now trained by an objective that cannot be satisfied by a constant (§1.7.3).

1.10 Geometric optimisation-based translation recovery

Translation estimation differs fundamentally from rotation estimation. Determining orientation requires semantic understanding of geometry and fragment relationships; determining translation, once orientations are known, is primarily a geometric consistency problem. Translations are therefore not predicted by the network but obtained by solving an optimisation.

Let p_i and q_i denote corresponding interface points from mutual nearest-neighbour matching.

$$\begin{aligned}
 E_{\text{pos}} &= \sum_i \|p_i - q_i\|^2 && \text{matching surfaces should occupy the same locations,} \\
 E_{\text{normal}} &= \sum_i \|n_i^{(A)} + n_i^{(B)}\|^2 && \text{mating surfaces should have opposing normals,} \\
 E_{\text{collision}} &&& \text{penalises volume interpenetration.}
 \end{aligned}$$

The translation vectors are recovered globally by

$$\min_{t_1, \dots, t_N} \left(E_{\text{pos}} + \lambda_{E_1} E_{\text{normal}} + \lambda_{E_2} E_{\text{collision}} \right),$$

placing fragments so as to satisfy interface alignment, normal consistency and collision avoidance simultaneously.

This stage is specified but not implemented. It is stated here as unchanged from V4 so that the design remains complete, but no result in this document depends on it, and §1.13 records it as outstanding.

1.11 Workflow: training, validation, and inference

1.11.1 The shared path

Training and validation share almost the entire pipeline. A scene directory is read once into fragments in their assembled frame; the fracture mask is computed from geometry; coincidence clusters and the ground-truth copy are taken in the assembled frame; a random rotation is drawn per fragment; the scene is standardised and tokens are sampled; fragments are concatenated into one ragged batch. The network runs identically in both modes. The path diverges only at the end: training computes the objective and steps the optimiser, while validation runs the same forward pass without gradients and scores the prediction.

Two ordering constraints are absolute. **Coincidence must be computed before perturbation**, because it is defined in the assembled frame; afterwards nothing coincides and the labels come back empty while looking like a result. And the ground-truth and perturbed copies must be built once and split, for the topology reason of §1.3.6.

1.11.2 Validation is not label-free

The perturbation is generated by the pipeline itself, so $Q^\top$ is known and every metric is computable. What validation measures is generalisation to unseen *objects*, not to unknown labels.

A defect found here is worth recording, because it is the worst shape a defect can take. Official split matching originally used only the object directory name, discarding the category. Breaking Bad lists objects as `category/object` and directory names are *not* unique across categories, so sixteen objects appeared in both the training and validation splits. Validation then becomes partly a memorisation test, and the number it produces is *better* than the honest one — it reads as success. Matching is now on the pair, and the loader refuses to run if any two splits intersect.

1.11.3 A real broken object

A third regime is worth separating explicitly, because it is the one the thesis must be honest about. On a genuinely broken object rather than a Breaking Bad sample:

- there is no Q , hence no label — a prediction exists and nothing checks it;
- there are no coincidence labels, so the dihedral mask alone selects tokens, at precision 0.24 (§1.5.2). GARF’s answer is a trained fracture segmenter reaching above 99.5%; this project does not have one;
- a rotation does not place a fragment — stage two is required and is not built.

Any figure reported on Breaking Bad is therefore an *optimistic bound*, because it uses ground-truth fracture labels a deployment would not have.

1.12 Implementation within a commodity hardware budget

The economy question of §1.1 makes implementation a design concern rather than an afterthought, so the constraints that shaped it are recorded here.

The binding constraint is memory, and it was in an unexpected place. Cross-fragment attention gathers query, key and value tensors to the *pair* dimension. Measured as the slope against pair count, that retained **1,109 bytes per pair**: at 2,048 tokens over six fragments — roughly 3.5 million pairs — **3.6 GB for a single scene**, per layer. Recomputing those gathers during the backward pass instead of storing them costs one extra forward pass of an indexing operation and takes the figure to **86 bytes per pair, 0.28 GB**, a thirteen-fold reduction with outputs and gradients bitwise identical.

A retraction belongs here. The token budget of 2,048 per scene was originally justified by comparing pair counts against GARF’s stack — a *floating-point* argument, made about hardware with far more memory. It was never checked against the memory of the target device, which is the binding constraint and is quadratic in exactly that number. The value survives, but it survived by luck rather than because the check had been performed.

Gradient accumulation is not a free substitute for a larger batch. When memory forces the batch down, accumulation is the standard remedy, but dividing each micro-batch’s loss by the accumulation count weights each *scene* equally, while a true batch weights each *fragment* equally — and a scene holds between two and 35 fragments. Measured on a two-fragment and an eight-fragment scene, the two gradients had cosine similarity **0.80** and norms 45% apart: a silent re-weighting of the objective introduced by a setting chosen for memory reasons. Weighting each micro-batch by its fragment count and normalising by the group total makes the two identical to seven decimal places on all four geometric terms. The contrastive term cannot be made to

match, because its negatives are drawn from the batch — and should not be, since within-scene negatives are the confusion set that matching actually faces.

Sessions are capped, so a run is a chain. Training must survive being killed at a fixed wall-clock limit: checkpoints are written on a timer rather than only at epoch boundaries, because a single epoch over the full training set can outlast a session; the always-current checkpoint is kept separate from the best-so-far, because a policy-gated file alone can freeze while training continues and a resume then silently discards the gap; and the learning-rate schedule is a pure function of the step count rather than a stateful object, because such objects are periodic past their horizon and store that horizon in their own state.

1.13 Status, and what has not been established

This section exists because the failure mode of this class of system is a plausible number rather than an exception.

1.13.1 Established

The data and geometry layer, the equivariant network, the composite objective and the training engine are implemented and covered by an automated test suite that runs clean under warnings-as-errors. Equivariance holds to the residuals of §1.4.5. The rotation convention is asserted by round trip rather than inferred. The architecture demonstrably *can* fit: driven on a small fixed set it reduces rotation error from 113° to 0.4° on one scene and from 137° to 0.6° on three; on 32 scenes it reached 13.3° after 1,608 steps and was still descending. These are *training*-error results and prove nothing about generalisation — which is exactly what makes them useful, since they isolate “are the conventions self-consistent” from “does it generalise”.

1.13.2 Not established

Nothing has been trained to convergence on the real dataset. Every figure in this document is a property of the architecture, the data, or the implementation — not evidence that the system learns the task. A short calibration run over two full epochs left the rotation error flat at chance, which at roughly 3,500 of a planned 65,000 optimiser steps is not yet evidence either way.

1.13.3 Open questions

- **Inference-time labelling.** Coincidence is unavailable at test time and dihedral is weak (§1.5.2). Until a segmenter exists, reported numbers are an optimistic bound.
- **Supervision asymmetry.** The correspondence loss is defined over coincident vertices, so it supervises only the fracture channel. Tokens drawn from the original surface are shaped by the rotation loss alone.
- **Loss weights, layer counts, token budget.** All are currently chosen rather than measured. The weights are deliberately equal so that the first runs measure their natural scales.
- **Fragments with no fracture surface.** Coincidence reports a minimum of zero, which should be impossible; most likely single-fragment modes, but it needs counting, because such fragments receive no inter-fragment edges at all.
- **The translation solver.** Specified in §1.10, not implemented.